

# Suffix Tree

## 后缀树

后缀树是一种维护一个字符串所有后缀的数据结构。

## 一些记号

记构建后缀树的母串为  $S$ ，长度为  $n$ ，字符集为  $\Sigma$ 。

令  $S[i]$  表示  $S$  中的第  $i$  个字符，其中  $1 \leq i \leq n$ 。

令  $S[i:j]$  表示  $S$  中第  $i$  个字符至第  $j$  个字符组成的字符串，称为  $S$  的一个子串。

记  $S[i:n]$  为  $S$  的以  $S[i]$  开头的后缀，为  $S[1:i]$  的以  $S[i]$  结尾的前缀。

## 定义

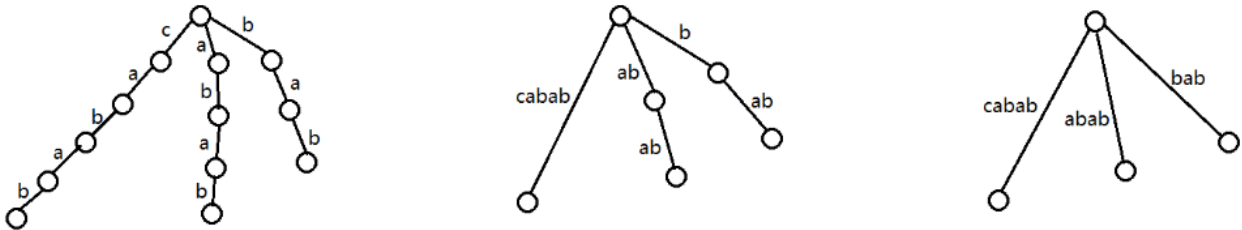
定义字符串  $S$  的 **后缀 trie** 为将  $S$  的所有后缀插入至 trie 树中得到的字典树。在后缀 trie 中，节点  $x$  对应的字符串为从根节点走到  $x$  的路径上经过的字符拼接而成的字符串。记后缀 trie 中所有对应  $S$  的某个后缀的节点为后缀节点。

容易看出后缀 trie 的优越性质：它的非根节点恰好能接受  $S$  的所有本质不同非空子串。但构建后缀 trie 的时空复杂度均为  $O(n^2)$ ，在很多情况下不能接受，所以我们引入后缀树的概念。

如果令后缀 trie 中所有拥有多于一个儿子的节点和后缀节点为关键点，定义只保留关键点，将非关键点形成的链压缩成一条边形成的压缩 trie 树为 **后缀树 (Suffix Tree)**。如果仅令后缀 trie 中所有拥有多于一个儿子的节点和叶结点为关键点，定义只保留关键点形成的压缩 trie 树为 **隐式后缀树 (Implicit Suffix Tree)**。容易看出隐式后缀树为后缀树进一步压缩后得到的结果。

在后缀树和隐式后缀树中，每条边对应一个字符串；每个非根节点 对应了一个字符串集合，为从根节点走到 的父亲节点 经过的字符串，拼接上至 的树边对应的字符串的任意一个非空前缀，称为 **前缀**。同时，在隐式后缀树中，称一个没有对应任何节点的后缀为 **隐式后缀**。

下图从左至右分别为以字符串  $S = \text{cabab}$  为母串构建的后缀 trie、后缀树和隐式后缀树。



考虑将  $S$  的后缀逐个插入至后缀 trie 中。从第二次插入开始，每次最多新增一个拥有多于一个儿子的节点和一个后缀节点，所以后缀树中节点个数最多为  $n$  个，十分优秀。

## 后缀树的建立

### 支持前端动态添加字符的算法

反串建 SAM 建出的 parent 树就是这个串的后缀树，所以我们将反串的字符逐个加入 SAM 即可。

▼ 参考实现

```

1 struct SuffixAutomaton {
2     int tot, lst;
3     int siz[N << 1];
4     int buc[N], id[N << 1];
5
6     struct Node {
7         int len, link;
8         int ch[26];
9     } st[N << 1];
10
11 SuffixAutomaton() : tot(1), lst(1) {}
12
13 void extend(int ch) {
14     int cur = ++tot, p = lst;
15     lst = cur;
16     siz[cur] = 1, st[cur].len = st[p].len + 1;
17     for (; p && !st[p].ch[ch]; p = st[p].link) st[p].ch[ch] = cur;
18     if (!p)
19         st[cur].link = 1;
20     else {
21         int q = st[p].ch[ch];
22         if (st[q].len == st[p].len + 1)
23             st[cur].link = q;
24         else {
25             int pp = ++tot;
26             st[pp] = st[q];
27             st[pp].len = st[p].len + 1;
28             st[cur].link = st[q].link = pp;
29             for (; p && st[p].ch[ch] == q; p = st[p].link) st[p].ch[ch] = pp;
30         }
31     }
32 }
33 } SAM;

```

## 支持后端动态添加字符的算法

Ukkonen 算法是一种增量构造算法。我们依次向树中插入串 的每一个字符，并在每一次插入之后正确地维护当前的后缀树。

### 朴素算法

首先介绍一种较为暴力的构建方式，我们用字符串 来演示一下构建的过程。

初始建立一个根节点，称为 号节点。同时每条边我们维护一个区间 表示这条边上的字符串为 。另外，维护已经插入的字符个数， 初始为 。

首先插入字符，直接从 号节点伸出一条边，标为， 指向一个新建的节点。这里的 是一个极大值，可理解为串的结尾，这样在插入新字符时，这条边会自动的包含新的字符。

□

接下来我们插入字符，同样从 伸出一条边，标为。注意到之前延伸出的边 的意义自动地发生了变化，随着串结尾的改变，其表示的串从 变为了。这样是正确的，因为之前所有后缀都已经以一个叶节点的形式出现在树中，只需要向所有叶节点的末端插入一个当前字符即可。

□

接下来，我们要再次插入一个字符，但是 是之前已经插入的字符串的一个子串，因此原树已经包含，此时，我们什么都不做，记录一个 表示 是当前最长的隐式后缀。

□

接下来我们插入另一个。因为前一个 没有插入成功，此时， 代表要插入的后缀为。我们从根开始向下寻找，发现也在原树之中。同样，我们还是什么都不做。

□

注意到我们没有管 之后的后缀。因为如果 是一个隐式后缀，那么对于， 都是隐式后缀。因为由 为隐式后缀可知，存在字符 使得 为 的子串，所以 也为 的子串，由隐式后缀树的定义可知 也不作为叶结点出现。

接下来我们插入，此时， 因此我们需要沿着根向下寻找，发现不在原树中。我们需要在 处代表的节点延伸出一条为 的出边。但发现这个节点其实不存在，而是包含在一条边中，因此我们需要分裂这条边，创建一个新节点，再在创建的节点处展现出我们要创建的出边。此时成功插入，令，因为 不再是隐式后缀。

□

接下来，因为 变化了，我们重复这个过程，直到再次出现隐式后缀，或（在这个例子中，是后者）。

构建过程结束。

该算法每次暴力从根向下寻找并插入的复杂度最坏为  $O(n)$ ，所以总的复杂度为  $O(n^2)$ 。

后缀链接

朴素算法慢主要是因为每次 `extend` 都要从根找到最长隐式后缀的插入位置。所以考虑把这个位置记下来。首先，我们采用一个二元组  $(p, l)$  来描述当前这个最长的被隐式包含的后缀。沿着节点  $p$  的开头为  $l$  的出边走长度  $l$  到达的位置应该唯一表示一个字符串，每次插入新的字符时，我们只需要从  $(p, l)$  和  $(q, l')$  描述的位置查找即可。

现在，我们只需要在  $(p, l)$  时更新。此时如果  $l < l'$ ，只需要让  $(p, l) = (q, l')$ ，因为下一个要插入的后缀是刚才插入的长度  $l'$ 。否则，设  $(p, l)$  对应的子串为  $s$ ，我们需要找到一个节点  $q$  对应  $s$ ，令  $(p, l) = (q, l')$  即可。

首先有引理：对隐式后缀树中任意非叶非根节点  $p$ ，在树中存在另一非叶节点  $q$ ，使得  $s$  是  $(q, l')$  对应的子串删去开头的字符。

证明。令  $s = a_1a_2\dots a_l$  表示删去开头字符形成的字符串。由隐式后缀树的定义可知，存在两个不同的字符  $a_1, a_2$ ，满足  $a_1a_2\dots a_l$  与  $a_1a_2\dots a_{l-1}a_2$  均为  $s$  的子串。所以， $a_1a_2\dots a_{l-1}a_2$  也为  $s$  的子串，所以在后缀 trie 中也对应了一个有分叉的关键点，即在隐式后缀 trie 中存在  $(q, l')$  使得  $s$  是其子串。证毕。

由该引理，我们定义  $link(p)$ ，称为  $p$  的**后缀链接 (Suffix Link)**。于是  $link(p)$  一定存在。现在我们只要能求出隐式后缀树中所有非根非叶节点的  $link$  即可。

Ukkonen 算法

Ukkonen 算法的整体流程如下：

为了构建隐式后缀树，我们从前往后加入  $S$  中的字符。假设根节点为  $r$ ，且当前已经建出  $S[1..i]$  的隐式后缀树且维护好了后缀链接。 $(p, l)$  的最长隐式后缀为  $s$ ，在树中的位置为  $q$ 。设  $c$  为  $S[i+1]$ ，现在我们需要加入字符  $c$ 。此时， $(p, l)$  的每一个后缀都需要在末尾添加字符  $c$ 。由于所有显式后缀都对应树中某个叶结点，它们父边右端点为  $q$ ，无需维护。所以，现在我们只用考虑隐式后缀末尾添加  $c$  对树的形态产生的影响。首先考虑  $l < l'$ ，有两种情况：

1.  $l < l'$  位置的转移。此时后缀树形态不会发生变化。由于  $(p, l)$  已经在后缀树中出现，所以对于  $(q, l')$ ，也会在后缀树中出现，此时只需将  $link(p)$  指向  $(q, l')$ ，不需做任何修改。
2.  $l > l'$  不存在  $(q, l')$  的转移。如果  $l = l'$  恰好为树中的节点，则此节点新增一条出边；否则需要对节点进行分裂，在此位置新增一个节点，并在新增节点处添加出边  $c$ 。此时对于  $(p, l)$ ，我们并不知道  $c$  会对后缀树形态造成什么影响，所以我们还需继续考虑  $link(p)$ 。考虑怎么求出  $link(p)$  在后缀树中的位置：如果  $l < l'$  不为  $link(p)$ ，可以利用后缀链接，令  $link(p) = link(link(p))$ ；否则，令  $link(p) = (q, l')$ 。最后令  $link(p) = link(link(p))$ ，再次重复这个过程。

每一步都只消耗常数时间，而算法在插入全部的字符后停止，所以时间复杂度为  $O(n)$ 。

由于 Ukkonen 算法只能处理出  $S$  的隐式后缀树，而隐式后缀树在一些问题中的功能可能不如后缀树强大，所以在需要时，可以在  $S$  的末端添加一个从未出现过的字符，这时  $S$  的所有后缀可以和树的所有叶子一一对应。

▼ 参考实现

```

1 struct SuffixTree {
2     int ch[M + 5][RNG + 1], st[M + 5], len[M + 5], link[M + 5];
3     int s[N + 5];
4     int now{1}, rem{0}, n{0}, tot{1};
5
6     SuffixTree() { len[0] = inf; }
7
8     int new_node(int s, int le) {
9         ++tot;
10        st[tot] = s;
11        len[tot] = le;
12        return tot;
13    }
14
15    void extend(int x) {
16        s[++n] = x;
17        ++rem;
18        for (int lst{1}; rem; ) {
19            while (rem > len[ch[now][s[n - rem + 1]])
20                rem -= len[now = ch[now][s[n - rem + 1]]];
21            int &v{ch[now][s[n - rem + 1]]}, c{s[st[v] + rem - 1]};
22            if (!v || x == c) {
23                lst = link[lst] = now;
24                if (!v)
25                    v = new_node(n, inf);
26                else
27                    break;
28            } else {
29                int u{new_node(st[v], rem - 1)};
30                ch[u][c] = v;
31                ch[u][x] = new_node(n, inf);
32                st[v] += rem - 1;
33                len[v] -= rem - 1;
34                lst = link[lst] = v = u;
35            }
36            if (now == 1)
37                --rem;
38            else
39                now = link[now];
40        }
41    }
42 } Tree;

```

## 作用

后缀树上每一个节点到根的路径都是 的一个非空子串，这在处理很多字符串问题时都很有用。

后缀树的 DFS 序就是后缀数组。后缀树的一个子树也就对应到后缀数组上的一个区间。后缀树上两个后缀的最长公共前缀是它们对应的叶节点的 LCA，因此，后缀数组的 height 的结论可以理解为树上若干个节点的 LCA 等于 DFS 序最小的和最大的节点的 LCA。

## 例题

### [洛谷 P3804【模板】后缀自动机（SAM）](#)

题意：

给定一个只包含小写字母的字符串。

请你求出 的所有出现次数不为 的子串的出现次数乘上该子串长度的最大值。

- 解法
- 参考代码

### [CF235C Cyclical Quest](#)

题意：给定一个小写字母主串 和 个询问串，求每个询问串 的所有循环同构在主串中出现的次数总和。

- 解法
- 参考代码

## 参考文献

1. 2021 国家集训队论文《后缀树的构建》代晨昕

本页面最近更新：2023/9/24 17:56:54, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [Eletary](#), [lr1d](#), [megakite](#), [Tiphereth-A](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用